

cases have received the lists of balanced parentheses strings after adding an opening and/or closing parenthesis (if possible).

Let's ask the three recursive algorithm questions about the `getBalancedParens()` function:

What is the base case? When the number of opening and closing parentheses remaining to be added to the string being built has reached 0. A second base case always occurs after the recursive cases have finished.

What argument is passed to the recursive function call? The total number of pairs of parentheses (`pairs`), the remaining number of opening and closing parentheses to add (`openRem` and `closeRem`), and the string currently being built (`current`).

How does this argument become closer to the base case? As we add more opening and closing parentheses to `current`, we decrement the `openRem` and `closeRem` arguments until they become 0.

The *balancedParentheses.py* file contains the Python code for our balanced parentheses recursive function:

```
def getBalancedParens(pairs, openRem=None, closeRem=None, current='', indent=0):
    if openRem is None: ❶
        openRem = pairs
    if closeRem is None:
        closeRem = pairs

    print('.' * indent, end='')
    print('Start of pairs=' + str(pairs) + ', openRem=' +
          str(openRem) + ', closeRem=' + str(closeRem) + ', current="' + current + '"')
    if openRem == 0 and closeRem == 0: ❷
        # BASE CASE
        print('.' * indent, end='')
        print('1st base case. Returning ' + str([current]))
        return [current] ❸

    # RECURSIVE CASE
    results = []
    if openRem > 0: ❹
        print('.' * indent, end='')
        print('Adding open parenthesis.')
```